Hyperparameter Optimization and Complete Production Case Study

From Experimentation to Production-Ready LLM

LECTURE 8 | DURATION · 60 MIN | EXPERIMENTS → PRODUCTION

TOPICS COVERED

- HPO Loop
- Objective Functions
- Search Space
- Manual Tuning
- Grid Search
- Random Search
- Bayesian Optimization
- Early Stopping
- Hyperband & ASHA
- Experiment Tracking
- Reproducibility
- Pareto Frontier
- Production Case Study
- Monitoring & Feedback



WHERE ARE WE IN THE COURSE?

From Tuning by Trial to Tuning by Design

LECTURE 1
LLM Foundations

→

LECTURE 2
Pre-training, Prompting, RAG, Fine-Tuning

→

LECTURE 3
Fine-Tuning Process + Taxonomy

→

LECTURE 4
PEFT → LoRA → QLoRA

↓

LECTURE 5
Inference + Latency

→

LECTURE 6
Compression

→

LECTURE 7
Hyperparameters

→

LECTURE 8 YOU ARE HERE
Hyperparameter Optimization + Complete Production Case Study

Lecture 7 explained the knobs.

Lecture 8 explains how to tune them.

HPO fundamentals → Search methods → Experiment design → Complete case study → Module recap

SECTION 01 · HPO FUNDAMENTALS

Why Do We Need Hyperparameter Optimization?

CONFIGURATION A

LR = 1e-5 | Epochs = 2 | Rank = 4

RESULT

Val F1 = 0.71

VS

CONFIGURATION B

WINNER

LR = 2e-4 | Epochs = 5 | Rank = 8

RESULT

Val F1 = 0.91

SAME: dataset | model | task

DIFFERENT: hyperparameters

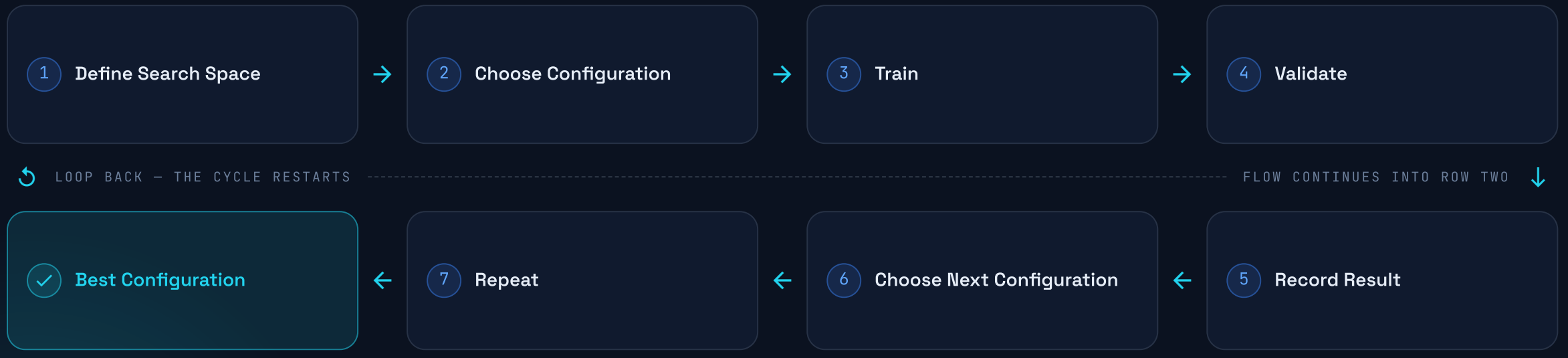
Very different performance.

A strong model with poor hyperparameters may perform worse than a smaller model with a good configuration.

SECTION 01 · DEFINITION

What Is Hyperparameter Optimization?

Hyperparameter Optimization (HPO) is the **systematic process** of searching for hyperparameter values that produce the **best validation performance** under **specified constraints**.



conceptually — every HPO method follows this skeleton

SECTION 01 · OBJECTIVE FUNCTION

Before Optimizing — Define the Objective

⚠ Do not begin searching until you know what **"best"** means.

POSSIBLE OBJECTIVE

Maximize Validation Macro-F1

BUT PRODUCTION MAY REQUIRE

Maximize F1 *subject to:*

- GPU Memory ≤ 20 GB
- TTFT ≤ 1 s
- Model Size ≤ 5 GB

! Optimization requires an objective, not merely a collection of experiments.

SINGLE VS MULTI-OBJECTIVE

SINGLE-OBJECTIVE — max Validation F1 — *simple.*

MULTI-OBJECTIVE

we may care about:

- Quality ↑
- Latency ↓
- Memory ↓
- Cost ↓

EXAMPLE — TWO CANDIDATE MODELS

Model A	F1 = 0.92	Latency = 300 ms
Model B	F1 = 0.90	Latency = 90 ms

"Which is better?" → **Depends on production requirements.**

SECTION 01 · SEARCH SPACE

The Search Space — and How It Explodes

SEARCH SPACE

For an illustrative LoRA fine-tuning experiment:

Learning Rate	1e-5 → 5e-4
Epochs	2, 3, 5
Batch Size	2, 4, 8
LoRA Rank	4, 8, 16, 32
LoRA Alpha	8, 16, 32, 64
LoRA Dropout	0.0, 0.05, 0.1
Sequence Length	512, 1024, 2048

These are example search ranges, not universal recommended values.

COMBINATORIAL EXPLOSION

5 learning rates × 3 batch sizes × 4 ranks × 4 alpha values × 3 dropout values × 3 epoch values

$$5 \times 3 \times 4 \times 4 \times 3 \times 3 = 2160 \text{ experiments}$$

if one run takes 2 hours → 2160 × 2 = **4320 GPU-hours**

"Should we try every combination?"

Usually no. This motivates smarter HPO.

SECTION 02 · SEARCH METHODS

Manual Tuning

WHERE VALUES COME FROM

the human selects values based on:

- experience
- previous experiments
- literature
- intuition
- training curves

EXAMPLE RUN LOG

```
Run 1: LR = 1e-4
  ↓
Run 2: LR = 2e-4
  ↓
Run 3: LR = 5e-4
```

each run informs the next guess

ADVANTAGES

- \+ simple
- \+ useful for initial understanding

LIMITATIONS

- – subjective
- – difficult to reproduce
- – inefficient for large search spaces

SECTION 02 · SEARCH METHODS

Grid Search

THE GRID choose fixed values, test every cell

LR ↓ \ Rank →	4	8	16
1e-4	●	●	●
2e-4	●	●	●
5e-4	●	●	●

Total: 3 × 3 = 9 experiments

ADVANTAGES

- \+ simple
- \+ systematic
- \+ reproducible
- \+ useful for small search spaces

PROBLEMS

- – expensive
- – scales poorly
- – wastes trials on unimportant dimensions
- – continuous variables must be discretized

! Grid Search becomes impractical when many LLM hyperparameters interact.

SECTION 02 · SEARCH METHODS

Random Search

Instead of testing every combination: **randomly sample configurations** from the search space.

EXAMPLE TRIALS

```
Trial 1: LR = 1.7e-4 · Rank = 8 · Dropout = 0.05
Trial 2: LR = 4.3e-5 · Rank = 16 · Dropout = 0.0
Trial 3: LR = 2.8e-4 · Rank = 4 · Dropout = 0.1
```

irregular, information-rich samples

WHY IT WORKS WELL

Suppose performance mainly depends on: `Learning Rate`

but less on: `Dropout`

Grid Search may waste many runs repeating similar LR values

vs

Random Search explores more distinct values.

For large search spaces, random search can be much more resource-efficient than exhaustive grid search.

SECTION 02 · COMPARISON

Grid Search vs Random Search

PROPERTY	GRID SEARCH	RANDOM SEARCH
Selection	predetermined	sampled
Coverage	regular	irregular
Large spaces	expensive	✓ better
Continuous variables	awkward	✓ natural
Simplicity	very high	high
LLM tuning	limited small search	✓ often useful baseline

→ neither method learns from previous trials — that is the next slide's problem

SECTION 02 · SMARTER SEARCH

Bayesian Optimization

Instead of choosing trials blindly:

THE LOOP

01 Previous Experiments

↓

02 Build Model of Hyperparameters → Performance

↓

03 Estimate promising region

↓

04 **Choose Next Trial**

↓

05 Evaluate

↓

06 Update Search Model

BACK TO TOP

Use results from previous experiments to decide what to try next.

THE FOG ANALOGY

Imagine searching for the **highest mountain in fog**.

RANDOM SEARCH

Try random locations.

BAYESIAN OPTIMIZATION

1 Try a location → 2 Observe altitude

3 Update belief about landscape → 4 Choose promising next location

Fewer expensive training runs may be needed.

SECTION 02 · SMARTER SEARCH

Exploration vs Exploitation — and Stopping Bad Trials Early

Optimization must balance:

EXPLORATION

Try **uncertain regions**.

Maybe something better exists here.

EXPLOITATION

Try **near the current best**.

This region already looks promising.

Good HPO balances both.

EARLY STOPPING OF BAD TRIALS

TRIAL A (good) — Good.

Epoch 1	F1 0.65
Epoch 2	F1 0.79
Epoch 3	F1 0.88

TRIAL B (unpromising)

Epoch 1	F1 0.42
Epoch 2	F1 0.43

If clearly unpromising: **why spend another three hours?**

Terminate it early.

SECTION 02 · SMARTER SEARCH

Successive Halving, Hyperband and ASHA

CONCEPT — SUCCESSIVE HALVING / HYPERBAND

Start many configurations
↓
Give each a small budget
↓
Remove poor performers

→

Give survivors more budget
↓
Remove again
↓
Best configurations receive largest training budget



! **Do not spend equal compute** on obviously poor trials.

ASHA = Asynchronous Successive Halving Algorithm

Run trials **independently** and stop poor-performing trials without waiting for all trials to reach the same stage.

USEFUL WHEN: training jobs take different times | jobs run on multiple GPUs / nodes | jobs are expensive

Keep this conceptual.

TOOLS FOR HPO

Examples you may encounter:

`Optuna` `Ray Tune` `Weights & Biases Sweeps` `Hyperopt` `scikit-learn search utilities` `Cloud HPO services`

★ The optimization concept matters more than the specific software tool. **The tool can change; the experimental principles do not.**

SECTION 03 · EXPERIMENT DESIGN

Every Trial Must Be Recorded

TRIAL RECORD For every run:

×15 FIELDS PER RUN

Run ID	Model	Dataset Version	Random Seed	Learning Rate
Batch Size	Epochs	Sequence Length	Rank	Alpha
Dropout	Validation Metric	Training Time	GPU Memory	Checkpoint

WHY?

Without experiment tracking: you may find a good model **and later not know how you created it**.

*tracking tools help — **but the discipline is the point***

SECTION 03 · EXPERIMENT DESIGN

Reproducibility and Random Seeds

REPRODUCIBILITY — RECORD:

- code version
- package versions
- model revision
- dataset version
- random seed
- hardware
- preprocessing
- hyperparameters
- evaluation procedure

A result you cannot reproduce is difficult to trust or deploy.

RANDOM SEED

Training includes **randomness**:

- parameter initialization
- dropout
- data shuffling
- sampling

*two identical configurations may show **slightly different results***

PRACTICE FOR IMPORTANT EXPERIMENTS

run multiple seeds

↓

report **Mean ± Variation**

↓

rather than overinterpreting one lucky run

SECTION 03 · EVALUATION DISCIPLINE

Validation Selects — the Test Set Confirms

CORRECT

TRAIN → learn model parameters

↓

VALIDATION → select hyperparameters

↓

TEST → one final unbiased evaluation

WRONG

Try configuration → Check test → Change configuration → Check test again

*The test set gradually becomes **part of development**.*

Hyperparameter optimization must not repeatedly optimize against the test set.

AVOID OVER-OPTIMIZING THE VALIDATION SET

Even the validation set can become **indirectly overfit** if hundreds of decisions are all based on one small validation set.

POSSIBLE PRACTICES

- larger validation data
- cross-validation for suitable small-data tasks
- multiple validation slices
- final untouched test
- robustness checks

SECTION 03 · PRODUCTION OBJECTIVES

Cost-Aware HPO and the Pareto Frontier

COST-AWARE OBJECTIVES

Do **not** optimize accuracy alone.

$$\text{Score} = F1 - \lambda \cdot (\text{GPU Cost})$$

conceptually

OR USE CONSTRAINTS:

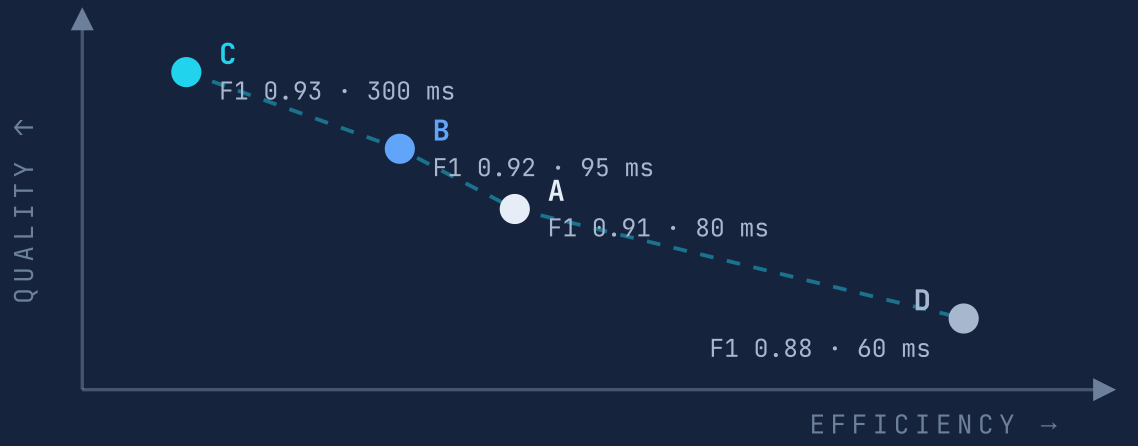
- GPU Memory ≤ 24 GB
- Training Time ≤ 2 hr
- Inference p95 ≤ 1.5 s

Best validation accuracy is not necessarily the best production configuration.

PARETO FRONTIER

MODEL	F1	LATENCY
Model A	0.91	80 ms
Model B	0.92	95 ms
Model C	0.93	300 ms
Model D	0.88	60 ms

A **Pareto-efficient point**: you cannot improve one objective without worsening another.



Production optimization is often multi-objective.

SECTION 04 · COMPLETE PRODUCTION CASE STUDY

Case Study — University Helpdesk LLM

THE TASK

Build an assistant that converts student complaints into:

- Department
- Category
- Priority
- Suggested Action

INPUT

"My semester fee was deducted but the portal still shows unpaid."

↓

OUTPUT

DEPARTMENT	CATEGORY
Accounts	Payment Reconciliation
PRIORITY	**ACTION**
High	Verify transaction reference

1 STEP 1 · DEFINE REQUIREMENTS

FUNCTIONAL:

- ✓ understand student query
- ✓ classify department
- ✓ classify issue
- ✓ assign priority
- ✓ generate action
- ✓ return structured output

NON-FUNCTIONAL (ILLUSTRATIVE TARGETS):

- Macro-F1 ≥ 0.90
- Structured output validity ≥ 98%
- p95 latency ≤ 2 s
- GPU memory ≤ available limit

Production begins with measurable requirements.

SECTION 04 · STEPS 2-3

Prompt Baseline First — then the RAG Question

2 STEP 2 · PROMPT BASELINE

Base Model + Detailed Prompt → Evaluation

MACRO-F1

0.68

LOW FOR PRODUCTION

VALID JSON

74%

1 IN 4 BREAKS

illustrative baseline results

Prompting alone is insufficient. Now fine-tuning is justified.

3 STEP 3 · DO WE NEED RAG?

"What is the current deadline for examination registration?" — This changes. → **Use: RAG**

"Classify this query into university department / category." — This is behavior. → **Use: Fine-Tuning**

RAG	→	current university information
FINE-TUNING	→	consistent internal behavior

a combined production system may use both

SECTION 04 · STEPS 4-5

Dataset Engineering and the Base Model

4 STEP 4 · DATASET ENGINEERING

DATASET CONTAINS:

- Instruction
- User Query
- Expected Structured Output

USER: "My hall ticket is unavailable."

ASSISTANT: Department: Examination · Category: Hall Ticket · Priority: High · Action: Verify eligibility.

ENSURE:

- ✓ label consistency
- ✓ phrase diversity
- ✓ balanced classes
- ✓ deduplication
- ✓ train/val/test split
- ✓ no leakage

5 STEP 5 · CHOOSE BASE MODEL

Qwen2.5-0.5B-Instruct

POSSIBLE REASONS:

- manageable size
- instruction-tuned
- suitable for laboratory experimentation
- lower hardware requirement

In production, model selection should compare multiple candidates.

SECTION 04 · STEPS 6-7

Fine-Tuning Strategy and the Initial Hyperparameter Space

STEP 6 · CHOOSE FINE-TUNING STRATEGY

OPTIONS

Full Fine-Tuning | **LoRA** | **QLoRA**

Suppose GPU is limited. → **Choose: LoRA / QLoRA**

WHY?

- fewer trainable parameters
- smaller adapter checkpoints
- lower training-memory requirement

STEP 7 · INITIAL HYPERPARAMETER SPACE

Learning Rate	{5e-5, 1e-4, 2e-4, 5e-4}
Epochs	{2, 3, 5}
LoRA Rank	{4, 8, 16}
Alpha	{8, 16, 32}
Dropout	{0, 0.05, 0.1}
Sequence Length	{512, 1024}

Again: illustrative — not universal.

SECTION 04 · STEP 8

First HPO Stage — and the Trial Results

START INEXPENSIVE | For example: Random Search | 10-20 trials | Goal: identify promising region.

CONTROL: fixed dataset | fixed model | same evaluation metric | controlled seed | comparable training budget

TRIAL	LR	EPOCHS	RANK	DROPOUT	VAL F1
T1	1e-4	3	8	.05	.86
T2	2e-4	3	8	.05	.90
T3	5e-4	3	8	.05	.77
T4	2e-4	5	8	.05	.92
T5	2e-4	8	8	.05	.89
T6 ★	2e-4	5	16	.05	.921

THE QUESTION *"Would you automatically choose T6?"* Not necessarily — T6: 0.921 vs T4: 0.920, but T6 has: more trainable parameters | more adapter memory →

Production may prefer T4.

SECTION 04 · STEPS 9-10

Inspect the Curves — then Select the Configuration

STEP 9 · INSPECT TRAINING CURVES

FOR BEST TRIALS INSPECT:

- Training Loss
- Validation Loss
- Validation F1
- Learning Rate
- Gradient Norm

UNDERFITTING — Train poor · Val poor

GOOD FIT — Train improves · Val improves

OVERFITTING — Train improves · Val worsens

⚠ *HPO metrics alone should not replace curve inspection.*

STEP 10 · SELECT BEST VALIDATION CONFIGURATION

LR = 2e-4	Epochs = 5
Rank = 8	Alpha = 16
Dropout = 0.05	best quality-resource trade-off

✕ **Do NOT claim:** *"These are the best universal values."*

✓ **Say:** *"These are the best values for this experiment according to our validation objective."*

SECTION 04 · STEPS 11-12

The Final Test — and Error Analysis

STEP 11 · FINAL TEST

— EVALUATE EXACTLY ONCE ON THE UNTOUCHED TEST SET

Test Accuracy	= 0.92
Macro-F1	= 0.91
Structured Output Validity	= 99%

Base Macro-F1 = 0.68 → **Fine-Tuned = 0.91**

Now we have evidence of improvement.

STEP 12 · ERROR ANALYSIS

— DO NOT STOP AT ONE METRIC

INSPECT: Confusion Matrix · Per-Class Precision · Per-Class Recall · Failure Examples



"Which class deserves investigation?" → **Student Services.**

SECTION 04 · STEPS 13-14

Compression Decision and Inference Benchmark

STEP 13 · COMPRESSION DECISION

Model quality is acceptable. Now ask: *"Can deployment be cheaper?"*

FP16 INT8 INT4 → Benchmark all.

VARIANT	F1	MEMORY	P95 LATENCY
FP16	.91	1.0 GB	550 ms
INT8	.908	.55 GB	470 ms
INT4	.89	.32 GB	430 ms

If requirement: F1 ≥ .90 → INT8 may be chosen over INT4.

Compression choice is requirement-driven.

STEP 14 · INFERENCE BENCHMARK

MEASURE:

TTFT TPOT End-to-End Latency Throughput p50 p95 p99 GPU Memory

USE REALISTIC PROMPT LENGTHS:

Short query Medium query Long query and realistic concurrency

Training benchmark ≠ production benchmark.

SECTION 04 · STEPS 15-16

Adding RAG — and the Production Architecture

STEP 16 · PRODUCTION ARCHITECTURE

- Student / Client
- API Gateway
- Authentication
- Input Validation
- Retriever ⇄ Vector / Document Store
- Prompt Construction
- **LLM Inference Server** — fine-tuned model + LoRA adapter
- Output Validation
- Response
- **Logging / Monitoring** — continuous loop

STEP 15 · ADD RAG

Student Query
↓
Retriever
↓
Current University Circular
↓
Prompt
↓
Fine-Tuned LLM
↓
Structured Answer

DIVISION OF LABOR

- **LoRA** → behavior
- **RAG** → current information

this diagram is the module in one picture

SECTION 04 · STEPS 17-18

Production Monitoring and the Feedback Loop

Deployment is not the end.

MONITOR · STEP 17 | FEEDBACK · STEP 18

MONITOR

SYSTEM

- Latency
- Throughput
- GPU utilization
- Memory
- Error rate

MODEL

- Output validity
- Task quality
- Hallucination / factuality where relevant
- Class distribution
- Failure examples

DATA

- Input distribution
- New terminology
- Concept drift

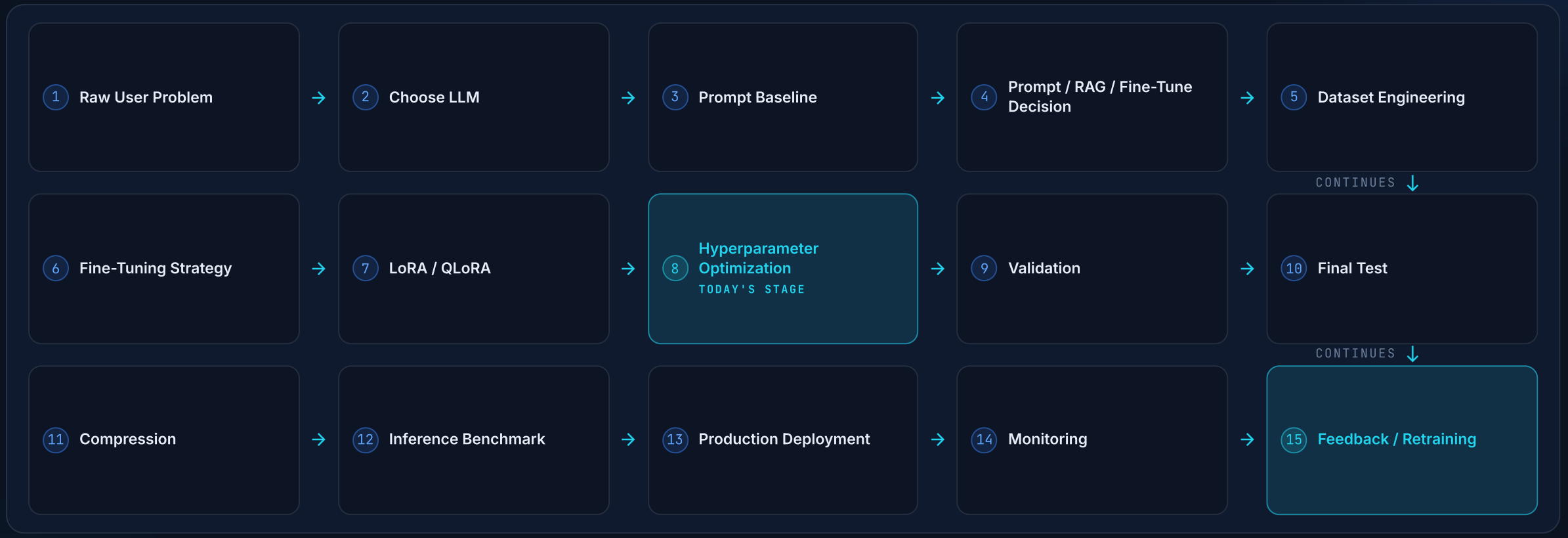
STEP 18 · FEEDBACK LOOP

1. Real Requests
2. Failures
3. Human Review
4. New Clean Examples
5. Dataset Version 2
6. Retrain / Fine-Tune
7. Evaluate
8. **Redeploy**

∞ Production ML is a lifecycle, not a one-time training event.

SECTION 05 · MODULE SYNTHESIS

The Complete Module Pipeline



COMPLETE EXPERIMENT RECORD

Every production candidate should have:

`Model ID` `Dataset Version` `Code Commit` `Hyperparameters` `Seed` `Hardware` `Training Duration` `Validation Metrics` `Test Metrics` `Model Size` `GPU Memory` `TTFT` `TPOT` `p95` `Throughput` `Deployment Version`

The model artifact without its experimental context is incomplete.

SECTION 05 · DECISION MAKING

One Decision Framework — Applied

ASK THESE QUESTIONS IN ORDER

1. Does prompting already solve it? → If yes: stop.
2. Is the problem missing / current knowledge? → Use: RAG.
3. Is consistent learned behavior needed? → Consider: fine-tuning.
4. Is full FT too expensive? → Use: PEFT / LoRA / QLoRA.
5. Which hyperparameters? → Use: validation-driven HPO.
6. Deployment too heavy? → Use: compression.
7. Serving too slow? → Measure: TTFT / TPOT / throughput / queueing.

CLASSROOM HPO EXERCISE

CONFIG	F1	GPU MEM	P95 LATENCY
A	.94	22 GB	2.3 s
B	.92	14 GB	1.4 s
C	.90	8 GB	0.9 s
D	.88	6 GB	0.7 s

REQUIREMENTS: F1 ≥ .90 · GPU Memory ≤ 16 GB · p95 ≤ 1.5 s

- ✓ B — yes
- ✓ C — yes
- ✗ A — memory and latency fail
- ✗ D — quality fails

"B or C — which one is best?" → Depends on whether the application values higher quality or lower cost / latency — multi-objective engineering judgment.

SECTION 05 · MODULE FINALE

Eight Lectures in Eight Lines

- L1 How does an LLM work?
- L2 When should we prompt, retrieve or fine-tune?
- L3 How does fine-tuning work?
- L4 How can we fine-tune efficiently?
- L5 How does LLM serving work?
- L6 How can we make models lighter?
- L7 Which configuration choices control training?
- L8 How do we systematically choose and deploy the best configuration? THIS LECTURE

A production LLM is not only a model.

Model + Data + Training + Evaluation + Serving + Infrastructure + Monitoring

"LLM production engineering is the discipline of optimizing the entire system, not merely maximizing one model metric."

Large Language Models: Production and Deployment — Module 1 Complete